

Hashcat e John the Ripper

Strumenti fondamentali per il password auditing e l'analisi della robustezza delle password

Versione revisionata — revisione delle citazioni, delle formulazioni e delle affermazioni dipendenti dalla versione degli strumenti

1. Introduzione

Le password a testo rimangono uno dei principali meccanismi di autenticazione nei sistemi informatici. La loro semplicità di implementazione e utilizzo le rende onnipresenti, ma le password scelte dagli utenti possono risultare vulnerabili a tentativi di recupero quando gli hash vengono ottenuti in un attacco offline. Per valutare la robustezza di un sistema di autenticazione, professionisti della sicurezza e amministratori di sistema utilizzano strumenti di password auditing, tra cui Hashcat e John the Ripper (JtR). Il presente saggio esamina la storia, l'architettura, le metodologie di attacco e le implicazioni per la sicurezza di questi due strumenti, basandosi sulla letteratura scientifica indicata in bibliografia e, per gli aspetti dipendenti dalle versioni correnti, sulla documentazione dei progetti.

2. John the Ripper: storia e funzionamento

John the Ripper è uno strumento open-source nato nell'ambito dei sistemi Unix e sviluppato originariamente da Solar Designer. Il progetto è stato concepito per l'auditing delle password e per l'individuazione di password deboli attraverso attacchi offline. L'edizione Jumbo, sviluppata con contributi della comunità, amplia considerevolmente il numero di formati e funzionalità supportati.

Tra le modalità tradizionali di JtR rientrano il single crack, che può utilizzare informazioni associate all'account per generare candidati; la modalità wordlist, che applica dizionari e regole di trasformazione; e modalità incrementali/statistiche destinate a esplorare lo spazio delle possibili password secondo strategie diverse. L'architettura estensibile di JtR ha consentito nel tempo l'integrazione di numerosi formati di hash e di accelerazione hardware.

La letteratura ha inoltre studiato la parallelizzazione di John the Ripper. Sykes, Lin e Skoczen hanno proposto algoritmi basati su MPI per distribuire il carico di lavoro tra processori, ottenendo miglioramenti rispetto all'esecuzione originale in specifici esperimenti. Tali risultati mostrano il valore della distribuzione del lavoro, ma non devono essere interpretati come benchmark universali delle prestazioni di JtR.

3. Hashcat: cracking accelerato e parallelizzazione

Hashcat è progettato per sfruttare il calcolo parallelo e l'accelerazione tramite GPU. Il progetto supporta un ampio insieme di hash mode e numerose modalità di attacco, tra cui dictionary, rule-based, combinator, mask/brute-force e modalità ibride. Poiché l'elenco dei formati e le capacità del programma cambiano con le versioni, in un confronto riproducibile è opportuno indicare sempre la versione di Hashcat utilizzata.

La letteratura scientifica ha mostrato ripetutamente il vantaggio potenziale dell'elaborazione parallela per funzioni di hashing adatte all'accelerazione GPU. Murakami, Kasahara e Saito hanno studiato una versione parallelizzata di un password-cracking tool tramite GPGPU, evidenziando riduzioni molto rilevanti del tempo di elaborazione in uno specifico scenario sperimentale. Analogamente, studi successivi hanno analizzato l'effetto dell'hardware GPU sul tasso di recupero delle password.

Brumen e Makari hanno studiato la resilienza delle password di studenti universitari e hanno mostrato che, in determinati dataset e condizioni sperimentali, una percentuale molto elevata delle password poteva essere recuperata mediante attacchi combinati. Il risultato è significativo come evidenza della prevedibilità delle password umane, ma non costituisce una misura universale della crackabilità di tutte le password.

4. Architettura dei sistemi di cracking

4.1 Modalità di attacco fondamentali

Gli attacchi di password cracking possono essere classificati in diverse categorie. Il brute-force esplora sistematicamente uno spazio di caratteri; l'attacco a dizionario utilizza una lista di candidati probabili; gli attacchi rule-based applicano trasformazioni sistematiche alle parole di base. Hashcat e John the Ripper permettono di combinare queste strategie in modi diversi.

Per una password di otto caratteri alfanumerici, assumendo 62 simboli disponibili e ammettendo la ripetizione, lo spazio teorico è 62^8 , pari a circa $2,18 \times 10^{14}$ combinazioni. Il tempo necessario per esplorarlo dipende tuttavia dall'algoritmo di hashing, dall'hardware, dalla modalità di attacco e da altri fattori; per questo una dimensione dello spazio di ricerca non può essere tradotta direttamente in un tempo universale di cracking.

Gli attacchi a dizionario possono essere potenziati sfruttando pattern osservati in password reali. Tatli ha mostrato che l'impiego di trasformazioni e pattern derivati da password reali può aumentare sensibilmente il numero di password recuperate rispetto a dizionari tradizionali in specifici dataset sperimentali.

4.2 Analisi delle regole di trasformazione

Liu et al. hanno introdotto tecniche di rule inversion e guess counting per analizzare il comportamento dei software di password cracking senza dover enumerare tutte le ipotesi generate. Queste tecniche permettono di studiare l'ordine e l'efficacia delle regole e di stimare la posizione di una password nello spazio delle guess.

Lo studio evidenzia inoltre l'importanza dell'ordinamento delle regole. Regole costruite manualmente da esperti possono avere un ordine subottimale quando confrontate con ordinamenti derivati empiricamente da dati di password reali. Il risultato suggerisce che gli approcci data-driven possono migliorare l'efficienza della ricerca.

4.3 Cracking distribuito

Con l'aumento del costo computazionale di algoritmi come PBKDF2, bcrypt, scrypt e Argon2, la distribuzione del carico può diventare rilevante in scenari di auditing autorizzato. Hranický et al. hanno studiato un sistema basato su BOINC e Hashcat, con strategie differenti per distribuire wordlist e keyspace. Lima et al. hanno successivamente proposto HaaS, una piattaforma per password cracking in sistemi distribuiti ed eterogenei, combinando containerizzazione e accesso a GPU locali e remote.

La letteratura ha anche analizzato scenari estremi in cui la capacità computazionale di grandi botnet viene applicata al password cracking. Queste stime sono utili per comprendere la superficie di minaccia, ma dipendono fortemente da ipotesi sull'hardware, sulla disponibilità dei nodi, sull'algoritmo di hashing e sull'efficienza effettiva del sistema distribuito.

5. Metodi avanzati di generazione delle ipotesi

5.1 Grammatiche probabilistiche context-free (PCFG)

Weir et al. hanno proposto l'uso di grammatiche probabilistiche context-free (PCFG) per generare strutture di password in ordine decrescente di probabilità. Addestrando il modello su password reali, il metodo ha mostrato, in specifici esperimenti, un miglioramento rispetto a JtR a parità di

numero di tentativi. Il risultato sostiene l'idea che la struttura delle password umane possa essere modellata statisticamente e sfruttata per ordinare le ipotesi in modo più efficace.

5.2 Metodi basati su pattern

L'analisi di password reali ha evidenziato pattern ricorrenti quali aggiunta di cifre, prefissi, inserimenti, ripetizioni, sequenze, sostituzioni di caratteri, inversioni, maiuscole e riferimenti a date o nomi. Tatli ha utilizzato queste strutture per costruire dizionari potenziati e ha osservato miglioramenti significativi in specifici dataset rispetto ai dizionari di riferimento.

5.3 Apprendimento automatico e reti neurali

Gli approcci di machine learning cercano di modellare direttamente la struttura delle password o di generare candidati secondo la probabilità appresa da grandi corpus. Studi recenti hanno combinato modelli strutturali e parallelizzazione multi-GPU, ottenendo miglioramenti rispetto a strategie di dizionario in specifici scenari sperimentali. Questi risultati mostrano una direzione di ricerca importante, ma le prestazioni non sono generalizzabili a tutti gli algoritmi di hashing o a tutti i dataset.

6. Ottimizzazione su GPU

6.1 Principi generali di ottimizzazione

L'esecuzione efficiente di algoritmi di hashing su GPU richiede ottimizzazioni sia algoritmiche sia architetturali. La letteratura ha studiato tecniche quali riduzione dei trasferimenti di dati, riutilizzo dei registri, istruzioni specializzate, precomputazione e strategie meet-in-the-middle. I risultati riportati in letteratura sono fortemente dipendenti da GPU, algoritmo, implementazione e versione del software utilizzati.

Studi recenti hanno inoltre analizzato l'ottimizzazione di PBKDF2-HMAC-SHA2 e del recupero di password per documenti Microsoft Office. I miglioramenti rispetto a Hashcat o ad altri software sono risultati misurabili in specifiche configurazioni hardware; per questo i valori percentuali devono essere letti come risultati sperimentali e non come caratteristiche permanenti dei programmi.

6.2 Gestione della memoria GPU

La gestione della memoria può avere un ruolo determinante nelle prestazioni degli algoritmi GPU. Hendarto e Kurniawan hanno analizzato diversi fattori prestazionali in un caso di studio relativo al brute-force di password PDF, rilevando un ruolo importante della memoria condivisa e ottenendo speedup significativi rispetto all'implementazione sequenziale in condizioni specifiche.

Cano et al. hanno inoltre studiato l'impiego dei Cuckoo filters per il matching tra hash generati e hash target, evidenziando vantaggi in termini di efficienza di memoria rispetto ad alcune strutture dati alternative. Anche in questo caso i risultati sono relativi allo scenario sperimentale considerato dagli autori.

7. Attacchi tramite botnet e cloud

La disponibilità di risorse cloud e di grandi reti compromesse aumenta potenzialmente la capacità di calcolo disponibile per gli attacchi offline. Le stime presenti in letteratura mostrano quanto rapidamente possa crescere la capacità di esplorazione dello spazio delle password quando il carico viene distribuito su grandi quantità di hardware. Tuttavia, confronti tra botnet e cloud devono tenere conto di costi, disponibilità delle risorse, latenza, efficienza GPU e limitazioni operative.

8. Difese contro il cracking delle password

8.1 Hashing sicuro

La difesa fondamentale contro il cracking offline consiste nell'utilizzo di password hashing progettato per rendere costoso il calcolo di ogni singola password candidata. Algoritmi come bcrypt, scrypt, PBKDF2 e Argon2 introducono costi computazionali e, in alcuni casi, requisiti di memoria che rendono più costosa la ricerca massiva rispetto a funzioni generiche come MD5 o SHA-1.

La scelta concreta dell'algoritmo e dei parametri deve essere effettuata secondo linee guida aggiornate e in funzione del sistema. I risultati di studi che confrontano CMS e framework devono essere interpretati nel loro periodo di osservazione e non automaticamente estesi alle versioni attuali dei prodotti.

8.2 Metriche di robustezza

La lunghezza della password è importante, ma non descrive da sola la difficoltà di recupero. Password costruite secondo pattern prevedibili possono risultare molto più facili da indovinare rispetto a stringhe della stessa lunghezza ma generate casualmente. La letteratura ha quindi proposto metriche e modelli alternativi alla sola entropia teorica di Shannon.

8.3 Politiche e controlli

Le politiche di sicurezza dovrebbero privilegiare lunghezza adeguata, password uniche, blocco delle password già compromesse e sistemi di autenticazione aggiuntivi. CAPTCHA, protocolli di autenticazione resistenti agli attacchi a dizionario, autenticazione multifattore e password manager possono contribuire a ridurre il rischio complessivo.

9. Implicazioni forensi e per la sicurezza

Il password cracking ha applicazioni legittime nell'informatica forense, nel penetration testing e negli audit di sicurezza, quando eseguito con autorizzazione. L'OSINT può aumentare l'efficacia della generazione di candidati perché informazioni pubbliche come nomi, date e interessi possono comparire nelle password; proprio per questo il loro impiego deve essere valutato nel rispetto del contesto autorizzativo, della privacy e della normativa applicabile.

Strumenti e toolkit che combinano crawling, profilazione, generazione di wordlist e attacchi offline possono essere utili agli amministratori per valutare la robustezza delle credenziali aziendali, purché utilizzati in ambienti controllati e con autorizzazione.

10. Conclusioni

Hashcat e John the Ripper rappresentano due strumenti importanti per il password auditing e sono caratterizzati da architetture e punti di forza differenti. Hashcat è fortemente orientato all'elaborazione parallela e all'accelerazione GPU; John the Ripper offre un'architettura estensibile, numerosi formati e modalità di analisi sviluppate nel corso della sua evoluzione. Un confronto corretto delle prestazioni richiede però di specificare algoritmo, versione, hardware, modalità di attacco e dataset.

La letteratura analizzata mostra inoltre che la prevedibilità delle password umane rimane un fattore critico. L'evoluzione delle GPU, del calcolo distribuito e dei metodi statistici aumenta la capacità di esplorare grandi spazi di ipotesi, rendendo essenziale l'adozione di funzioni di password hashing progettate per essere costose, l'uso di password uniche e robuste, l'autenticazione multifattore e un'adeguata formazione degli utenti.

Il futuro del password auditing combina sempre più modelli statistici, machine learning e calcolo eterogeneo. Sul fronte difensivo, la protezione efficace richiede un approccio multilivello che comprenda tecnologia, configurazione dei sistemi, gestione delle credenziali, policy e formazione.

Riferimenti bibliografici

- [1] E. Liu, A. Nakanishi, M. Golla, D. Cash, B. Ur, "Reasoning Analytically about Password-Cracking Software," IEEE Symposium on Security and Privacy, 2019, pp. 380–397. DOI: 10.1109/SP.2019.00070.
- [2] Openwall, "John the Ripper documentation and project information," documentazione ufficiale del progetto.
- [3] M.-D. Cano, A. Villafranca, I. Tasic, "Performance evaluation of Cuckoo filters as an enhancement tool for password cracking," Cybersecurity, vol. 6, 2023. DOI: 10.1186/s42400-023-00193-6.
- [4] P. Jourdan, E. Stavrou, "Towards Designing Advanced Password Cracking Toolkits," Adjunct Publication of the 27th Conference on User Modeling, Adaptation and Personalization, ACM, 2019, pp. 203–208. DOI: 10.1145/3314183.3324967.
- [5] E. R. Sykes, M. Lin, W. Skoczen, "MPI Enhancements in John the Ripper," Journal of Physics: Conference Series, vol. 256, 2010. DOI: 10.1088/1742-6596/256/1/012024.
- [6] R. Hranický, L. Zobal, O. Ryšavý, D. Kolář, "Distributed password cracking with BOINC and hashcat," Digital Investigation, vol. 30, 2019, pp. 161–172. DOI: 10.1016/j.diin.2019.08.001.
- [7] F. Yu, Y. Huang, "An Overview of Study of Password Cracking," 2015 International Conference on Computer Science and Mechanical Automation, 2015, pp. 25–29. DOI: 10.1109/CSMA.2015.12.
- [8] T. Murakami, R. Kasahara, R. Saito, "An implementation and its evaluation of password cracking tool parallelized on GPGPU," 2010 10th International Symposium on Communications and Information Technologies, 2010, pp. 534–538. DOI: 10.1109/ISCIT.2010.5665047.
- [9] B. Brumen, T. Makari, "Resilience of students' passwords against attacks," MIPRO, 2017, pp. 1275–1279. DOI: 10.23919/MIPRO.2017.7973619.
- [10] A. Kanta, S. Coray, I. Coisel, M. Scanlon, "How viable is password cracking in digital forensic investigation? Analyzing the guessability of over 3.9 billion real-world accounts," Forensic Science International: Digital Investigation, vol. 37, 2021, 301186. DOI: 10.1016/j.fsidi.2021.301186.
- [11] T. Kakarla, A. Mairaj, A. Y. Javaid, "A Real-World Password Cracking Demonstration Using Open Source Tools for Instructional Use," IEEE EIT, 2018. DOI: 10.1109/EIT.2018.8500257.
- [12] E. I. Tatli, "Cracking More Password Hashes With Patterns," IEEE Transactions on Information Forensics and Security, vol. 10, no. 8, 2015, pp. 1656–1665. DOI: 10.1109/TIFS.2015.2422259.
- [13] C. Lima, R. Alves, J. Rufino, "HaaS - A Platform for Password Cracking in Distributed Heterogeneous Systems," IEEE IPDPSW, 2025, pp. 866–875. DOI: 10.1109/IPDPSW66978.2025.00137.
- [14] J. A. Dev, "On the Imminent Advent of Botnet Powered Cracking," IEEE CIC, 2016, pp. 188–195. DOI: 10.1109/CIC.2016.035.
- [15] M. Weir, S. Aggarwal, B. de Medeiros, B. Glodek, "Password Cracking Using Probabilistic Context-Free Grammars," IEEE Symposium on Security and Privacy, 2009, pp. 391–405. DOI: 10.1109/SP.2009.8.
- [16] J. Galbally, I. Coisel, I. Sanchez, "A New Multimodal Approach for Password Strength Estimation—Part II: Experimental Evaluation," IEEE TIFS, vol. 12, no. 12, 2017, pp. 2845–2860. DOI: 10.1109/TIFS.2017.2730359.
- [17] X. Gan, D. Li, H. Chen, "Structural Rule-Driven Password Recovery with Multi-GPU Parallelization Techniques," IEEE ISPA, 2025, pp. 1407–1414. DOI: 10.1109/ISPA67752.2025.00194.
- [18] C. Ge et al., "Optimized Password Recovery for SHA-512 on GPUs," IEEE CSE/EUC, 2017, pp. 226–229.
- [19] D. Kim, S. C. Seo, "Efficient Optimization of MS Office 2013+ Password Cracking and PBKDF2-HMAC-SHA2 on GPUs," IEEE Access, vol. 12, 2024, pp. 96436–96448. DOI: 10.1109/ACCESS.2024.3426605.
- [20] I. L. S. Hendarto, Y. Kurniawan, "Performance factors of a CUDA GPU parallel program: A case study on a PDF password cracking brute-force algorithm," IC3INA, 2017, pp. 35–40. DOI: 10.1109/IC3INA.2017.8251736.
- [21] H. Metla, M. Vinay Reddy, C. Sai Kiran, V. Madhu Viswanatham, "Cryptanalysis of Secure Hash Password Technique (CSHPT) in Linux," SSRN Electronic Journal, 2018. DOI: 10.2139/ssrn.3167804.
- [22] C. Ntantogian, S. Malliaros, C. Xenakis, "Evaluation of password hashing schemes in open source web platforms," Computers & Security, vol. 84, 2019, pp. 206–224. DOI: 10.1016/j.cose.2019.03.011.
- [23] H. R. G. Cacacho, H. J. L. Manuel, E. L. Failoga, J. P. A. Acang, "Breaking the Password Security Standards Using Offline Attacks and Public User Attributes," IEEE HNICEM, 2019. DOI: 10.1109/HNICEM48295.2019.9072807.
- [24] R. Beno, R. Poet, "Hacking Passwords that Satisfy Common Password Policies," International Conference on Security of Information and Networks, ACM, 2020. DOI: 10.1145/3433174.3433616.
- [25] S. Chakrabarti, M. Singhal, "Password-Based Authentication: Preventing Dictionary Attacks," Computer, vol. 40, no. 6, 2007, pp. 68–74. DOI: 10.1109/MC.2007.216.
- [26] A. Kanta, I. Coisel, M. Scanlon, "A survey exploring open source Intelligence for smarter password cracking," Forensic Science International: Digital Investigation, vol. 35, 2020, 301075. DOI: 10.1016/j.fsidi.2020.301075.

Nota editoriale. Le percentuali, i benchmark e le capacità degli strumenti citati nel testo sono da interpretare nel contesto degli studi e delle versioni indicate. Per riprodurre un benchmark è necessario specificare hardware, software, algoritmo di hashing, parametri, dataset e modalità di attacco. L'uso degli strumenti descritti deve avvenire esclusivamente su sistemi e credenziali per i quali si dispone di autorizzazione.